

🕒 Cập nhật tháng 8 năm 2024

[Bài đọc] Cấu hình và triển khai JWT trong REST API

1. Cấu hình JWT trong REST API

- Để cấu hình JWT trong Spring Security cần thêm các thư viện cần thiết trong file pom.xml.

```

<!-- Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>

<!-- Json -->
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>

```

2. Triển khai JWT trong REST API

- Để token có thể thông qua các filter để thực hiện các request được client gửi về cần tạo 1 lớp được kế thừa từ lớp **OncePerRequestFilter** để ghi đè phương thức **doFilterInternal**:

```

public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private static final Logger LOGGER = LoggerFactory.getLogger(JwtAuthenticationFilter.class);

    private final JwtTokenProvider tokenProvider;

    private final CustomUserDetailsService customUserDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {
        try {
            String jwt = getJwtFromRequest(request);

            if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
                String userId = tokenProvider.extractUserIdFromJWT(jwt);

                UserDetails userDetails = customUserDetailsService.loadUserById(userId);
                UsernamePasswordAuthenticationToken authenticationToken = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
                authenticationToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

                SecurityContextHolder.getContext().setAuthentication(authenticationToken);
            }
        } catch (Exception ex) {
            LOGGER.error("Could not set user authentication in security context", ex);
        }
        filterChain.doFilter(request, response);
    }

    private String getJwtFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7, bearerToken.length());
        }
        return null;
    }
}

```

- Lớp này sẽ thực hiện việc lọc các token từ Client gửi về để xác thực token có hợp lệ hay không thông qua hàm validateToken. Nếu token hợp lệ thì cho phép thực hiện request gửi về, nếu token không hợp lệ sẽ đẩy ra thông báo lỗi đến cho Client.

```

public String extractUserIdFromJWT(String token) {
    Claims claims = Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJws(token).getBody();
    return String.valueOf(claims.getSubject());
}

public boolean validateToken(String token) {
    try {
        Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJws(token);
        return true;
    } catch (SignatureException ex) {
        LOGGER.error("Invalid JWT signature");
    } catch (MalformedJwtException ex) {
        LOGGER.error("Invalid JWT token");
    } catch (ExpiredJwtException ex) {
        LOGGER.error("Expired JWT token");
    } catch (UnsupportedJwtException ex) {
        LOGGER.error("Unsupported JWT token");
    } catch (IllegalArgumentException ex) {
        LOGGER.error("JWT claims string is empty");
    }
    return false;
}
}

```